

Predictions:

1. Experiment 3.1 — What a straight line can do

- a. What I will change (control):
 - i. Change the structure of the perceptron so that it has no hidden layer.
Perceptron:
 Input: 2 nodes
 Layer 1 (output, sigmoid): 1 node
 - ii. Held fixed:
 1. Learning rate: 0.05
 2. Error threshold: 0.0001
 3. Epochs trained: 5000
 4. Training data
 5. Validation data
 6. Weight initialization (random uniform from -1 to 1)
- b. What I predict will happen:
 - i. Predicted accuracy of 70%
- c. Why:
 - i. A single sigmoid node without a hidden layer can draw only a straight line through the input space. The two moons are curved, interleaved crescents, so no straight line can fully separate them, as it can only find the best fitting line and accept the rest as errors. The model prediction shape would be a single straight line cutting diagonally through the center of the plot. The misclassified points should cluster at the tips of the crescents, where each moon curls into the other class locations.

2. Experiment 3.2 — What the hidden layer buys

- a. What I will change:
 - i. Change the structure of the perceptron by adding a hidden layer of ReLU nodes between the input and the sigmoid output. Try different sizes for this hidden layer (2, 4, 8, 16 nodes).
Perceptron:
 Input: 2 nodes
 Layer 1 (hidden, ReLU): H nodes
 Layer 2 (output, sigmoid): 1 node
 - ii. Held fixed:
 1. Learning rate: 0.05
 2. Error threshold: 0.0001

3. Epochs trained: 5000
 4. Training data
 5. Validation data
 6. Weight initialization (random uniform from -1 to 1)
- b. What I predict will happen:
- i. Predicted accuracy of 95 - 100%
- c. Why:
- i. Each ReLU hidden layer node would add a linear fold to the decision boundary. Combining several folds would let the network build a bent, piecewise linear boundary instead of a single straight one. With enough hidden units, this bent boundary would be able to trace the curved shape of the two crescents. Thus, more nodes in the hidden layer it has would increase the accuracy of the model's prediction as the shape would become closer fit to the crescents.

3. Experiment 3.3 — Three ways to prevent training.

Structure: 1 hidden layer with 8 nodes (lowest average loss = 0.00537)

- a. What I will change:
- i. Weights to zero (initialize the hidden layer weights to zero, instead of random)
 - ii. Increase learning rate
 - iii. Decrease learning rate
- b. What I predict will happen:
- i. Accuracy similar to Experiment 3.2 (95 - 100)%
 - ii. Accuracy near 0 - 10 %
 - iii. Accuracy near 30 - 40 %
- c. Why:
- i. The weight starts at zero, but the network still computes gradients from the loss and updates every weight each epoch, so training should still proceed and eventually reach a good fit. Changing the weight to zero would just mean that it is starting at a less favorable point than random initialization.
 - ii. An extremely high learning rate would make each weight update overshoot the correct direction by a wide margin, so instead of settling toward good weights the network's outputs become unstable by a wide margin, causing it to get mostly incorrect.
 - iii. A learning rate that's too small makes each weight update tiny, so within the fixed number of epochs the network barely moves from its starting point and never reaches a usable decision boundary, creating a single horizontal line.

Experiment 3.1:

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.18452976722897266

=== Final Weights and Bias ===

Layer 1 weights:

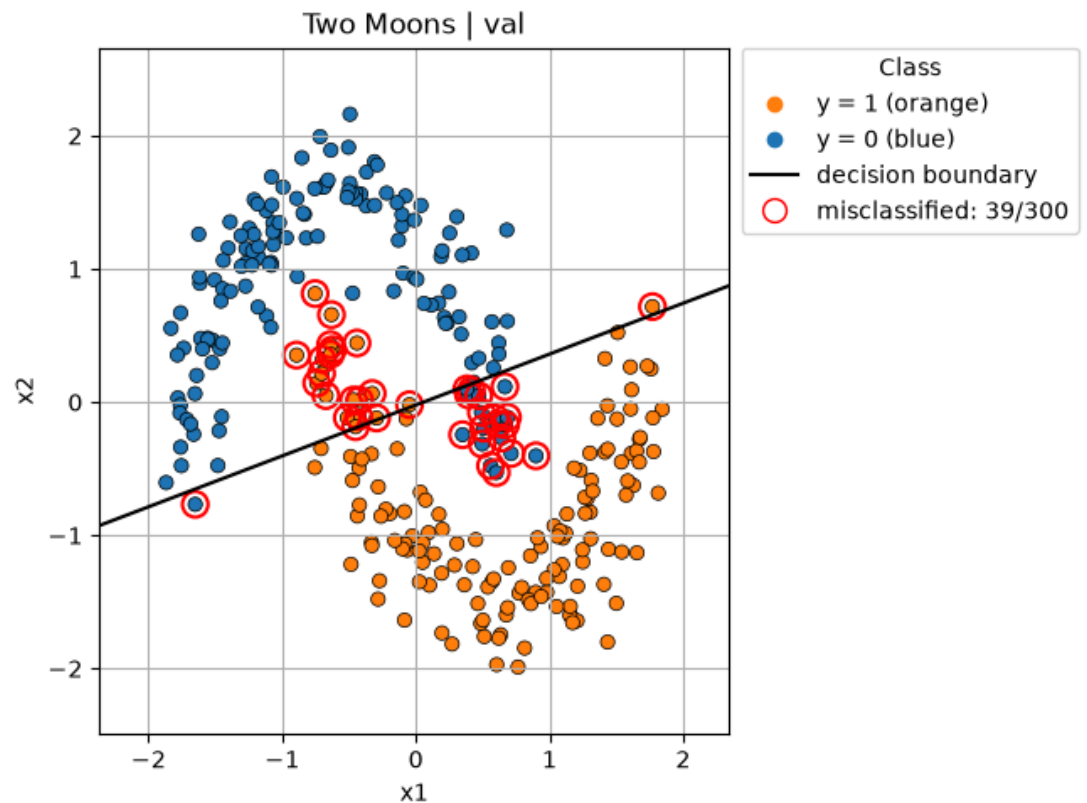
Node 1: [0.8874187634084348, -2.321134819167818]

Layer 1 bias: [-0.05031036201531784]

=== Validation Results ===

Misclassified / total: 39 / 300

Accuracy: 87.00%



Experiment 3.2:

2 nodes

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (hidden, sigmoid): 2 nodes

Layer 2 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.1682966473839296

=== Final Weights and Bias ===

Layer 1 weights:

Node 1: [-7.57488208377887, 0.7398701223830045]

Node 2: [0.049833988467618065, -1.3535357011314468]

Layer 1 bias: [-10.54429843585181, 0.6185513968933397]

Layer 2 weights:

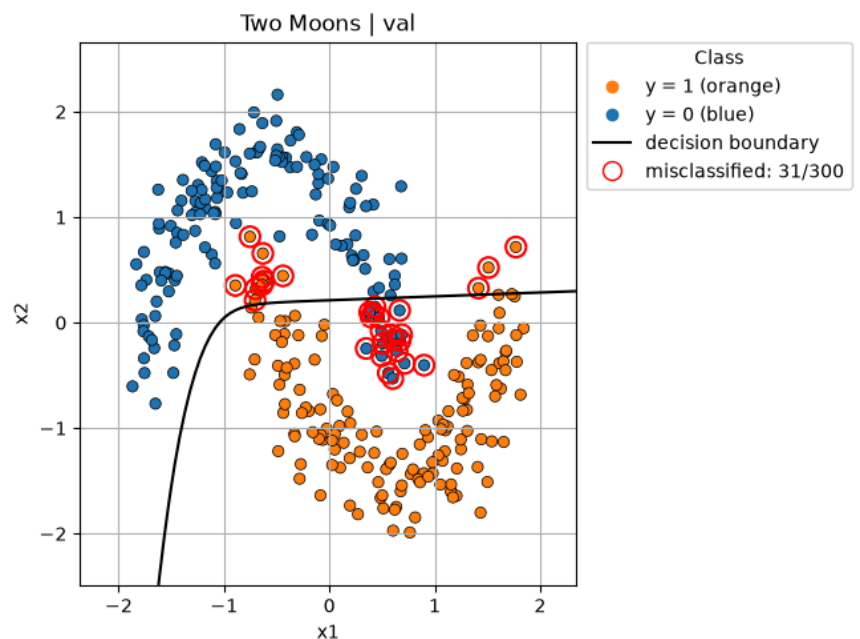
Node 1: [-6.978286227779941, 8.31554744435093]

Layer 2 bias: [-4.839402134054575]

=== Validation Results ===

Misclassified / total: 31 / 300

Accuracy: 89.67%



4 nodes

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (hidden, sigmoid): 4 nodes

Layer 2 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.006505691702408795

=== Final Weights and Bias ===

Layer 1 weights:

Node 1: [9.76016932095818, 3.3500571557140755]

Node 2: [-1.1793644188642871, -0.6012675416618116]

Node 3: [8.456035086855872, -4.02607982949334]

Node 4: [-8.755287595033158, 4.151037349771583]

Layer 1 bias: [0.2531841819914326, -2.196357363111894, -8.657728676890285, -9.171631119103187]

Layer 2 weights:

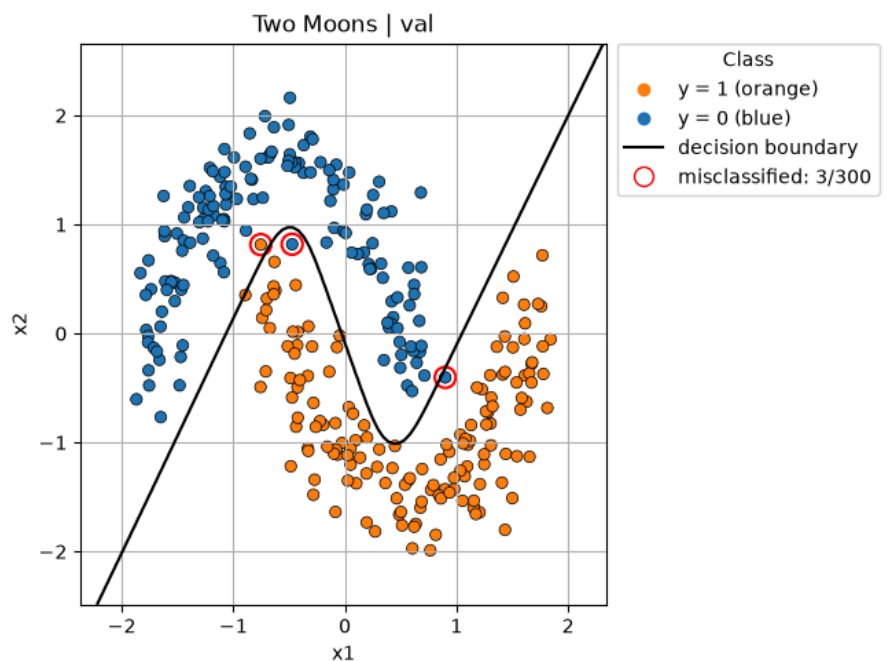
Node 1: [-14.253972247503865, -1.0524963416756583, 12.9642469445691, -12.678994177466825]

Layer 2 bias: [7.103508402925763]

=== Validation Results ===

Misclassified / total: 3 / 300

Accuracy: 99.00%



8 nodes

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (hidden, sigmoid): 8 nodes

Layer 2 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.005370228209392434

=== Final Weights and Bias ===

Layer 1 weights:

Node 1: [-4.526801318457095, 2.2152538552300354]

Node 2: [0.6421936341371438, 0.15465113130445537]

Node 3: [5.434726880431532, -2.9626773063970195]

Node 4: [10.239490918655298, 4.247549890868159]

Node 5: [-0.6410199982094498, -0.027596915369335676]

Node 6: [0.6130463656631704, -0.022288247051795614]

Node 7: [7.062737058004175, -3.2742718307733942]

Node 8: [-3.313144621144731, 1.4436174654798724]

Layer 1 bias: [-4.795511957462009, 0.5851906370601808, 5.9294561175657705,

-0.5129848198533495, -0.24796243348025498, -0.2685058935213087, -7.453557960618959,
-3.540770044980433]

Layer 2 weights:

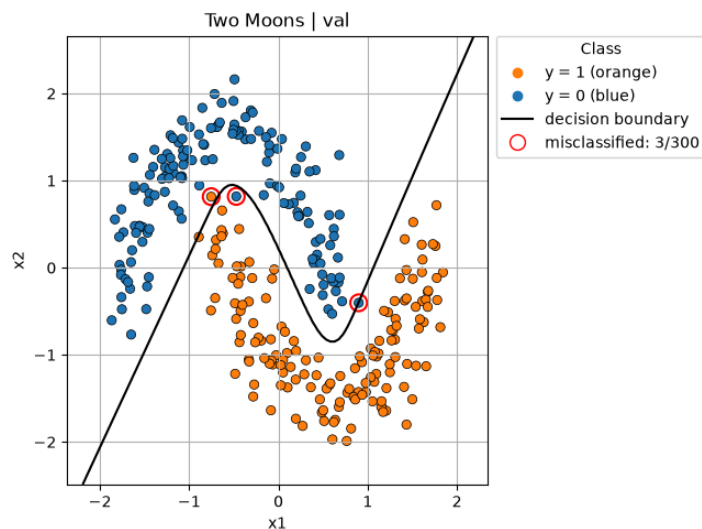
Node 1: [-7.195402608563773, 5.967604171982604, 8.454154356982874,
-20.822634496016068, -4.343174182596489, 4.785300339388587, 11.183370768994687,
-5.683443723657263]

Layer 2 bias: [0.3434657904384211]

=== Validation Results ===

Misclassified / total: 3 / 300

Accuracy: 99.00%



16 nodes

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (hidden, sigmoid): 16 nodes

Layer 2 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.009054993249575633

=== Final Weights and Bias ===

Layer 1 weights:

Node 1: [-1.001723894025618, 0.3777836453876068]

Node 2: [0.9977957404822606, -0.38878668267840955]

Node 3: [-1.0708298654470723, 0.38034325788654494]

Node 4: [-1.015460067207858, 0.3959668275600209]

Node 5: [-8.135906598839972, -2.525404433603524]

Node 6: [1.0187391236561996, -0.4078079644448756]

Node 7: [1.0263034384284786, -0.412353930890622]

Node 8: [-2.918317386739016, 1.449594339250197]

Node 9: [-1.1449483774634501, 0.4029098772714069]

Node 10: [1.851099145082618, -1.1107123858123489]

Node 11: [-1.152859628389638, 0.49518354618985844]

Node 12: [0.9957510587066568, -0.38887807612635544]

Node 13: [-0.9974040717920685, 0.37856635564582736]

Node 14: [-10.046150831993334, 2.1076125215540613]

Node 15: [1.110452460586445, -0.47296917068497324]

Node 16: [-1.0602947984003583, 0.43258829673128557]

Layer 1 bias: [0.04673925567732626, -0.2181165915645726, -0.48319490572964857, 0.28606085555472704, -0.21591966358804132, -0.3937676973920457, -0.4282045769488238, -3.0715079895108004, -0.7158693971502492, -2.1114930980756466, 0.8426649530983538, -0.21585533299713774, 0.07433058787511793, 0.24058809716070942, -0.7529229286481773, 0.5536297322386667]

Layer 2 weights:

Node 1: [-6.788481697863278, 3.5385064738271925, -6.104592869880914, -6.865385334622451, 15.929338242260908, 3.635874759347153, 4.0058162886738575,

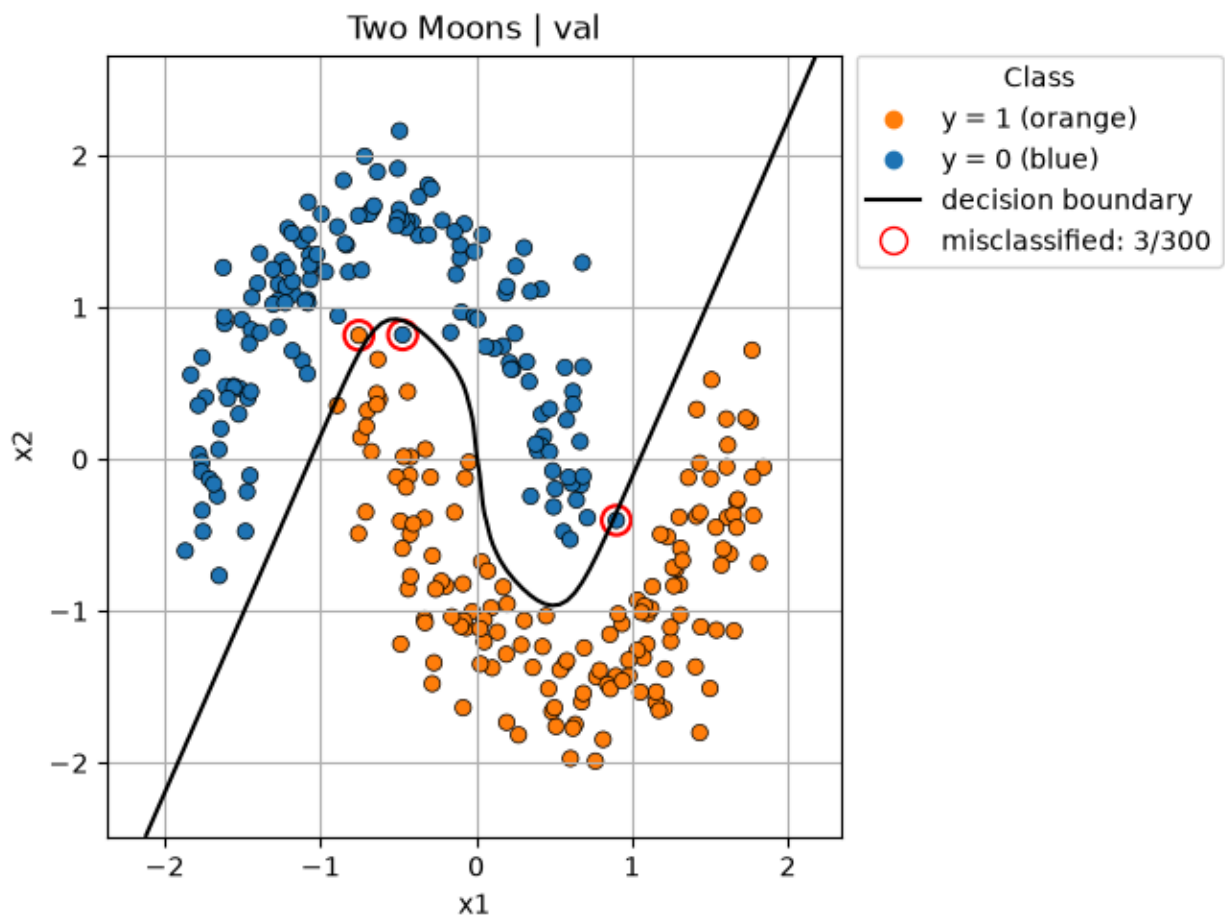
-7.210738805669388, -4.436569200007406, 4.765401192004626, -6.844187258672652,
3.1334641651273314, -5.01893676047712, 18.87475905402974, 4.1901360979467865,
-5.627338903822781]

Layer 2 bias: [-2.601637688666162]

=== Validation Results ===

Misclassified / total: 3 / 300

Accuracy: 99.00%



Experiment 3.3:

- Initial weights to zero

=== Model Structure ===

Perceptron:

Input: 2 nodes

Layer 1 (hidden, sigmoid): 8 nodes

Layer 2 (output, sigmoid): 1 node

=== Training Configuration ===

Learning rate: 0.05

Error threshold: 0.0001

Epochs trained: 5000

Average loss at end of training: 0.007099546077894943

=== Final Weights and Bias ===

Layer 1 weights:

Node 1: [9.590759989345942, 3.1676433958294234]

Node 2: [8.050053751006079, -4.268786206695981]

Node 3: [-0.3009722561336398, 2.3102879946218144]

Node 4: [-8.014635431307397, 4.102850331351219]

Node 5: [-0.8823616513065171, 0.0598406757842793]

Node 6: [-2.4467511197458824, 0.005337839735579009]

Node 7: [-3.8381256477479857, -2.3064040387860767]

Node 8: [1.6825587120450514, 0.14114631035846692]

Layer 1 bias: [0.2887398636887274, -8.475243065976606, 0.1944552887865831, -8.360274055728917, -1.2367531829589957, -2.7633161740734797, -0.2988097670671824, -1.7682130375410192]

Layer 2 weights:

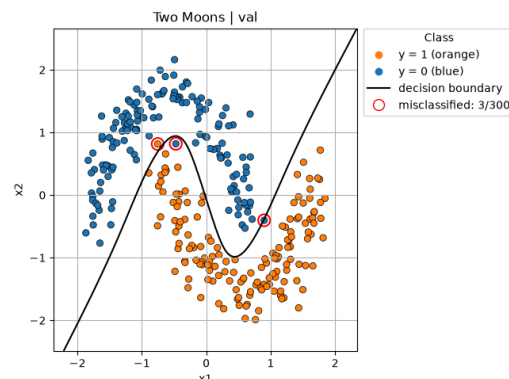
Node 1: [-12.399234120847938, 12.299071064879865, 3.5850012980634727, -12.44215007498112, -0.7570595057398354, -2.6388235383335936, 4.118971716824321, 3.205741915274463]

Layer 2 bias: [2.575001373333196]

=== Validation Results ===

Misclassified / total: 3 / 300

Accuracy: 99.00%



- Increasing learning rate

Run	Learning Rate	Max Epoch	Weight	Misclassified	Average Error	Accuracy (Percentage)
1	0.1	5000	Random	3/300	0.0029	99.00%
2	1.0	5000	Random	2/300	0.00065	99.33%
3	5.0	5000	Random	2/300	0.00030	99.33%
4	10.0	5000	Random	2/300	0.00018	99.33%
5	100.0	5000	Random	149/300	0.50	50.33%
6	1000.0	5000	Random	149/300	0.5	50.33%

- Decreasing learning rate

Run	Learning Rate	Max Epoch	Weight	Misclassified	Average Error	Accuracy (Percentage)
1	0.05	5000	Random	3/300	0.0054	99.00%
2	0.01	5000	Random	4/300	0.024	98.67%
3	0.005	5000	Random	4/300	0.047	98.67%
4	0.001	5000	Random	23/300	0.17	92.33%
5	0.0001	5000	Random	40/300	0.20	86.67%
6	0.00001	5000	Random	45/300	0.29	85.00%

Mechanism:

Experiment 3.1:

1. The trained perceptron settled on weights $[0.887, -2.321]$ with bias -0.050 , defining a single straight decision line: $0.887 \cdot x_1 - 2.321 \cdot x_2 - 0.050 = 0$. Because there is no hidden layer, this line is the only shape the model can ever produce. It can translate and change slope, but it can never bend. The optimizer found the orientation best splits the bulk of the two crescents (weighted more heavily toward x_2 , given the larger magnitude on that term), which is why it captured 87% of points correctly despite the data not being linearly separable. The remaining 39 misclassified points aren't random noise, but they are structurally forced errors, concentrated wherever the crescents curl across whatever single line the model settles on, since no straight boundary can ever correctly separate the interleaved tips.

Experiment 3.2:

1. Adding the hidden layer allowed the network to combine multiple linear boundaries (one per hidden node, from its own weights and bias) into a single nonlinear lines to combine, which is not enough independent folds to fully trace the curvature, so it underperformed even 3.1 model in some regions and lands at 89.67%. Once the hidden layer reaches 4 nodes, the network has enough independent lines available that Layer 2 can combine them into a boundary that closely follows the actual crescent shapes, dropping misclassifications to 3/300 (99.00% accuracy). Beyond 4 nodes (8, 16 nodes), accuracy did not improve any further, which meant that the extra hidden nodes ended up redundant or highly correlated with existing ones (visible in the 16 node weights, where several nodes have nearly identical weight vectors (like nodes 1, 3, 4, 13), meaning that the network already had enough capacity to fit the boundary and the additional units mostly add unused flexibility rather than new separating power.

Experiment 3.3:

1. With the prediction of getting stuck nearly at zero forever, the model reached 99.00% accuracy, matching the best result from 3.2 and not degrading. The reason the symmetry breaking failure did not show was that only the Layer 1 weights were zeroed, with the Layer 1 biases and the Layer 2 (output) weights were still randomly initialized. Since each hidden node connects to the output through a different, randomly-initialized Layer 2 weight, the error signal backpropagated to each hidden node was different from the very first epoch, even though all the nodes started with identical forward pass weights. That asymmetric downstream connection was enough to break the tie and let each node diverge onto a different feature during training. True symmetry only happens when every weight tied to a node is identical, including what feeds into it and what it feeds into.
2. The failure was not uniform across all “too high values”. It was a threshold effect. From 0.1 up through 10.0, accuracy held steady or improved (99.00% → 99.33% accuracy) with lower average error, because larger steps let the network converge faster within the

fixed 5000 epoch budget without overshooting badly. Only once the learning rate hit 100 did the updates become large enough to destabilize the weights entirely, with accuracy collapsing to 50.33% (which is the chance level) and average error holding steady at 0.5. The network's predictions saturated near 0.5 for every point rather than genuinely learning the boundary.

3. This matched the predicted mechanism partially, but the actual degradation was gradual. As the learning rate shrank from 0.05 to 0.00001, accuracy declined smoothly from 99.00% to 85.00%, with the average error rising correspondingly. This is consistent with the network still learning the correct direction, just too slowly to finish converging within the max epoch of 5000.

Reconciliation:

Experiment 3.1:

I predicted:

1. I predicted it to have about 70% accuracy with a diagonal boundary and errors clustered at the crescent tips.

What actually happened:

1. The model had 87.00% accuracy, with the boundary defined by weights [0.887, -2.321], with it still being a single diagonal line, consistent with the predicted shape. However, it classified noticeably more points correctly than expected.

What my mental model was missing:

1. I underestimated how much of the two-moons dataset can be separated by a linear line. Only the curled tips of each crescent actually crossed the line, so a single line captured more of the dataset correctly than expected.

Experiment 3.2:

I predicted:

1. I predicted around 95 to 100% accuracy, improving steadily as hidden layer size increases.

What actually happened:

1. The 2 hidden nodes reached only 89.67%. 4, 8, and 16 nodes all converged to the same 99.00%, but with 8 hidden nodes being the lowest error.

What my mental model was missing:

1. I assumed that the more hidden nodes the model has, it would draw a better fitting curve. Instead, there is a minimum capacity threshold. With 2 nodes wasn't enough to carve out the crescent boundary, but that threshold is crossed (around 4 nodes), extra capacity stops improving validation accuracy at all. The model plateaus rather than continuing to improve meaning the good enough point is reached much earlier than I predicted, and extra units mostly add unused flexibility rather than additional accuracy.